

TRƯỜNG ĐẠI HỌC ĐÀ LẠT

Khoa Công nghệ Thông Tin



BÀI TẬP NHÓM MÔN LẬP TRÌNH MẠNG

Đề tài: Xây dựng ứng dụng gửi tin nhắn (chat) Server-Client

Sinh viên thực hiện:

Nguyễn Ngọc Trường Dân – 2312590

Nguyễn Hưng Thịnh – 2312756

Võ Thị Minh Ân – 2312567

Lớp : CTK47A

Giảng viên : Trần Ngô Như Khánh

Lâm Đồng, tháng 12, năm 2025

Mục Lục

MỞ ĐẦU.....	1
Giới thiệu đề tài :	1
CHƯƠNG 1: MÔ HÌNH HOẠT ĐỘNG CỦA CHƯƠNG TRÌNH	2
1.1. Server cho phép nhiều Client kết nối cùng lúc.....	2
1.2. Client đăng nhập trước khi kết nối	2
1.3. Quản lý danh sách Client.....	2
1.4. Chức năng gửi tin nhắn cho tất cả Client (Broadcast)	2
1.5. Gửi tin nhắn trực tiếp giữa các Client	2
1.6. Nhóm chat	3
1.7. Tóm tắt luồng hoạt động của hệ thống Chat Server–Client	3
1.8. Sơ đồ hoạt động cơ bản của hệ thống Chat Server–Client.....	4
CHƯƠNG 2: CÁC KỸ THUẬT ,GIAO THỨC SỬ DỤNG.....	9
2.1. Công cụ hỗ trợ	9
2.2 Mô hình kiến trúc	9
2.3. Kiến trúc	9
2.6 Các kỹ thuật bổ sung	10
CHƯƠNG 3: MÔ TẢ CÁC LỚP VÀ PHƯƠNG THỨC CHÍNH	11
3.1 Lớp Server	11
3.2 Lớp Client Handler.....	12
3.2.1 Mục đích	12
3.2.2 Các thuộc tính chính	12
3.2.3 Các phương thức chính.....	13
3.2.4 Luồng hoạt động ClientHandler	13
3.2.5 Vai trò trong toàn hệ thống	14
3.3 Lớp Client.....	14
3.3.1 Mục đích	14
3.3.2 Các thuộc tính chính	14
3.3.3 Các phương thức chính.....	15
3.3.4 Luồng hoạt động của ChatClient	16
3.3.5 Vai trò trong toàn hệ thống	17

3.4 Lớp Message (tùy chọn).....	17
3.4.1 Mục đích	17
3.4.2 Các thuộc tính	17
3.4.3 Các phương thức chính.....	18
3.4.4 Vai trò trong hệ thống	19
3.4.5 Vị trí của lớp Message trong luồng hoạt động.....	19
CHƯƠNG 4: DEMO.....	20
4.1 Phía người dùng:	20
4.1.1 Giao diện đăng nhập và đăng ký:	20
4.1.2 Gửi tin nhắn trực tiếp giữa các Client:	20
4.1.3 Xem hồ sơ người dùng:	21
4.1.4 Tạo nhóm chat:	22
4.1.5 Nhắn tin khi người dùng offline:	24
4.2 Phía máy chủ (server):.....	25
4.2.1 Giao diện đăng nhập:.....	25
4.2.2 Giao diện quản lý danh sách client :.....	25
4.2.3 Chức năng gửi tin nhắn cho tất cả Client:	26
CHƯƠNG 6: KẾT QUẢ ĐẠT ĐƯỢC	27
6.1. Kết quả triển khai các chức năng cơ bản.....	27
6.1.1. Khả năng đa kết nối Client và đăng nhập xác thực.	27
6.1.2. Quản lý Trạng thái Client (Presence Management).....	27
6.1.3. Cơ chế trao đổi tin nhắn Server-Client	27
6.2. Kết quả triển khai các yêu cầu nâng cao	27
6.2.1. Gửi tin nhắn trực tiếp giữa các client	27
6.2.2. Tạo nhóm chat	28
6.2.3. Gửi tin nhắn ngoại tuyến	28
6.3. Tổng kết.....	28
TÀI LIỆU THAM KHẢO.....	29

DANH MỤC HÌNH ẢNH

Hình 1-1 Sơ đồ hoạt động cơ bản.....	4
Hình 1-2 Luồng hoạt động khi đăng nhập, gửi tin offline	5
Hình 1-3 Luồng gửi tin nhắn tới một user	6
Hình 1-4 Luồng gửi tin nhắn tới một group.....	7
Hình 1-5 Luồng gửi ảnh.....	8
Hình 4-1 . Form đăng nhập	20
Hình 4-2 . Form đăng ký.....	20
Hình 4-3 Hình ảnh gửi tin nhắn giữa client - client	21
Hình 4-4 Form profile	22
Hình 4-5 Hình ảnh tạo nhóm.....	23
Hình 4-6 Trước khi người nhận đăng nhập.....	24
Hình 4-7 Sau khi người nhận đăng nhập.....	24
Hình 4-8 Form quản lý client (server)	25
Hình 4-9 Gửi tin nhắn cho toàn bộ client.....	26

MỞ ĐẦU

Giới thiệu đề tài :

Trong thời đại Cách mạng công nghiệp 4.0, công nghệ thông tin giữ vai trò quan trọng trong mọi lĩnh vực, đặc biệt là kết nối và trao đổi dữ liệu. Các ứng dụng nhắn tin trực tuyến (chat) ngày càng phổ biến, trở thành công cụ thiết yếu trong học tập, làm việc và giao tiếp.

Xuất phát từ nhu cầu đó, nhóm chúng em thực hiện đề tài “Xây dựng ứng dụng gửi tin nhắn (chat) Server–Client” với mục tiêu nghiên cứu và xây dựng một ứng dụng giúp người dùng trao đổi thông điệp thông qua mô hình Client–Server, đảm bảo tốc độ, ổn định và bảo mật.

Đề tài có ý nghĩa cả về lý luận và thực tiễn: giúp sinh viên nắm vững nguyên lý hoạt động của mô hình Client–Server, cơ chế trao đổi dữ liệu, xử lý đa luồng, đồng thời rèn luyện kỹ năng phân tích, lập trình và làm việc nhóm.

Thông qua đề tài, sinh viên không chỉ củng cố kiến thức chuyên ngành, mà còn vận dụng lý thuyết vào thực tế, góp phần ứng dụng công nghệ phục vụ cuộc sống trong kỷ nguyên số hóa và kết nối toàn cầu.

CHƯƠNG 1: MÔ HÌNH HOẠT ĐỘNG CỦA CHƯƠNG TRÌNH

1.1. Server cho phép nhiều Client kết nối cùng lúc

Server được triển khai theo mô hình xử lý song song, sử dụng đa luồng (multithreading) hoặc cơ chế non-blocking I/O. Khi một Client gửi yêu cầu kết nối, Server sẽ tạo ra một luồng xử lý riêng hoặc một phiên làm việc (session) để giao tiếp với Client đó. Nhờ vậy, Server có thể lắng nghe và phục vụ nhiều Client đồng thời, đảm bảo hệ thống không bị nghẽn hoặc chặn khi một Client đang gửi hoặc nhận dữ liệu.

1.2. Client đăng nhập trước khi kết nối

Trước khi kết nối vào hệ thống chat, Client bắt buộc phải thực hiện quy trình đăng nhập. Người dùng cung cấp username và gửi lên Server để kiểm tra. Server xác minh rằng username hợp lệ và không trùng với bất kỳ Client nào đang trực tuyến. Nếu vượt qua bước kiểm tra này, Client sẽ được cấp quyền truy cập và có thể tham gia vào phòng chat hoặc sử dụng các chức năng của hệ thống.

1.3. Quản lý danh sách Client

Server duy trì một danh sách chứa thông tin của tất cả các Client kết nối vào hệ thống. Danh sách này bao gồm các Client đang online, tổng số Client đã kết nối, cùng các thông tin liên quan như username, socket, và trạng thái hoạt động. Mỗi khi có Client đăng nhập hoặc đăng xuất, danh sách sẽ được cập nhật.

1.4. Chức năng gửi tin nhắn cho tất cả Client (Broadcast)

Server hỗ trợ gửi tin nhắn đến tất cả Client đang online cùng lúc. Chức năng này thường được sử dụng trong các trường hợp như: gửi thông báo hệ thống, tin nhắn chung từ một Client tới toàn bộ người dùng, hoặc thông báo từ quản trị viên. Server đảm bảo rằng mọi Client đều nhận được tin nhắn broadcast ngay lập tức, duy trì tính đồng bộ và hiệu quả trong hệ thống.

1.5. Gửi tin nhắn trực tiếp giữa các Client

Hệ thống cho phép Client thực hiện gửi tin nhắn 1-1 đến một Client khác. Quy trình hoạt động như sau:

1. Client A chọn Client B và gửi tin nhắn.
2. Tin nhắn được chuyển lên Server.
3. Server thực hiện định tuyến và gửi tin nhắn đến đúng Client B.
4. Client B nhận và hiển thị tin nhắn trên giao diện.

Trong trường hợp Client B đang offline, tin nhắn sẽ được xử lý thông qua chức năng tin nhắn offline đảm bảo không bị mất dữ liệu.

1.6. Nhóm chat

Khi một Client gửi tin nhắn vào nhóm:

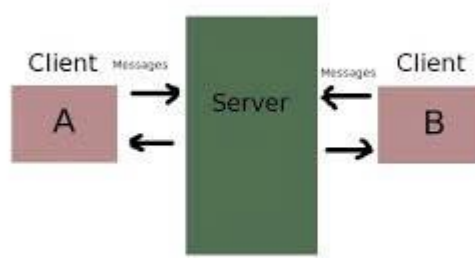
1. Server nhận tin nhắn từ Client.
2. Xác định nhóm tương ứng và danh sách thành viên trong nhóm.
3. Gửi tin nhắn đến tất cả các Client thuộc nhóm, đảm bảo mọi thành viên online đều nhận được thông tin.

Cơ chế này giúp hệ thống duy trì truyền thông nhóm hiệu quả, đồng bộ tin nhắn và quản lý quyền truy cập của các thành viên.

1.7. Tóm tắt luồng hoạt động của hệ thống Chat Server–Client

1. Đăng nhập: Client gửi thông tin đăng nhập → Server xác thực username.
2. Kết nối: Client đăng nhập thành công → được thêm vào danh sách Client online.
3. Gửi tin nhắn: Client gửi tin nhắn → Server xử lý và phân phối:
 - Broadcast: gửi tới tất cả Client online.
 - Trực tiếp: gửi tới một Client cụ thể.
 - Nhóm: gửi tới tất cả thành viên trong nhóm.
4. Tin nhắn offline: Nếu người nhận không online, tin nhắn được lưu → gửi khi Client đó kết nối lại.
5. Tạo nhóm chat: Client tạo nhóm → Server quản lý thành viên và tin nhắn → phát tin nhắn đến tất cả thành viên online.
6. Thoát hệ thống: Khi Client rời khỏi → Server cập nhật danh sách online để đảm bảo thông tin đồng bộ.

1.8. Sơ đồ hoạt động cơ bản của hệ thống Chat Server–Client



Hình 0-1 Sơ đồ hoạt động cơ bản

Luồng hoạt động chính:

- Để Client giao tiếp với Server hoặc yêu cầu Server thực hiện một nhiệm vụ, cụ thể như sau:

1. Client đóng gói yêu cầu thành chuỗi JSON (type,... các property đi kèm) với type là nhiệm vụ cần Server thực hiện.

Hình ảnh ví dụ:

```
await _tcp.SendAsync(new { type = "AUTH", username, password });
```

2. Server tiếp nhận, thực hiện tuần tự hóa, lấy firstline (được chia bởi \n).

```
dynamic obj = JsonConvert.DeserializeObject(firstLine);  
string messageType = (string)obj.type;
```

3. So sánh type đó trong switch case với các nhiệm vụ cụ thể được Client yêu cầu

```
if (messageType == "AUTH")  
{  
    string user = (string)obj.username;  
    string pass = (string)obj.password;  
  
    if (AuthManager.Validate(user, pass, out var acc))  
    {  
        Username = acc.Username;  
        Role = acc.Role;  
  
        OnlineRegistry.Add(this);  
        string avatarFileName = !string.IsNullOrEmpty(acc.Avatar) ? System.IO.Path.GetFileName(acc.Avatar) : "default.png";  
        await SendAsync(new { type = "AUTH_OK", username = Username, role = Role.ToString(), displayName = acc.DisplayName ?? acc.Username, avatar  
        // Sync existing groups for this user immediately after auth  
        try  
        {  
            var groups = GroupRegistry.ListByMember(Username);  
            foreach (var g in groups)  
            {  
                var msg = new { type = "GROUP_CREATED", groupId = g.GroupId, name = g.Name, members = g.Members };  
                await SendAsync(msg, ct);  
            }  
        }  
        catch { }  
    }  
}
```


a. Luồng hoạt động khi client đăng nhập, nhận tin offline

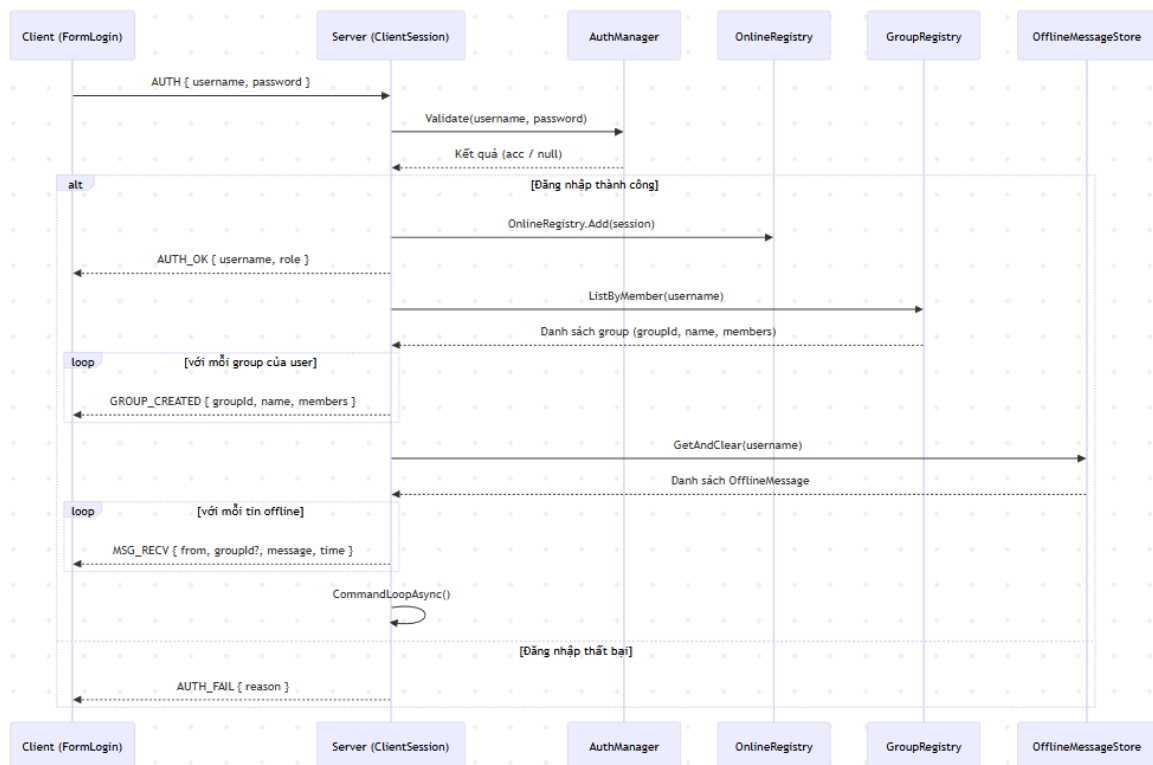
- Client yêu cầu phiên làm việc với server thông qua TcpClient và NetworkStream, server tạo phiên (dùng bất đồng bộ, thuận tiện cho việc multithread, hỗ trợ nhiều client kết nối cùng lúc) bằng TcpListener và TcpService.

+ Client: tạo chuỗi JSON, với firstline có thuộc tính type là “AUTH” gồm (type, username, password,...) kết thúc bằng \n (phân biệt giữa các chuỗi JSON)

+ Server thực hiện tuần tự hóa chuỗi JSON, lấy type và so sánh trong switch case

Nếu đúng, trả về cho client AUTH_OK – tiến hành đăng nhập, thực hiện vòng lặp (các chức năng, giao tiếp với server).

Nếu sai, trả về AUTH_FAIL



Hình 0-2 Luồng hoạt động khi đăng nhập, gửi tin offline

b. Luồng gửi tin nhắn tới một user

- Khi gửi tin nhắn tới Client khác, Server có nhiệm vụ trung gian, trung chuyển tin nhắn đến đầu cuối,.. cụ thể:

1. Client A nhắn tới Client B

2. A đóng gói thành chuỗi JSON với type là “MSG_TO”, gồm (type, message, clientb...

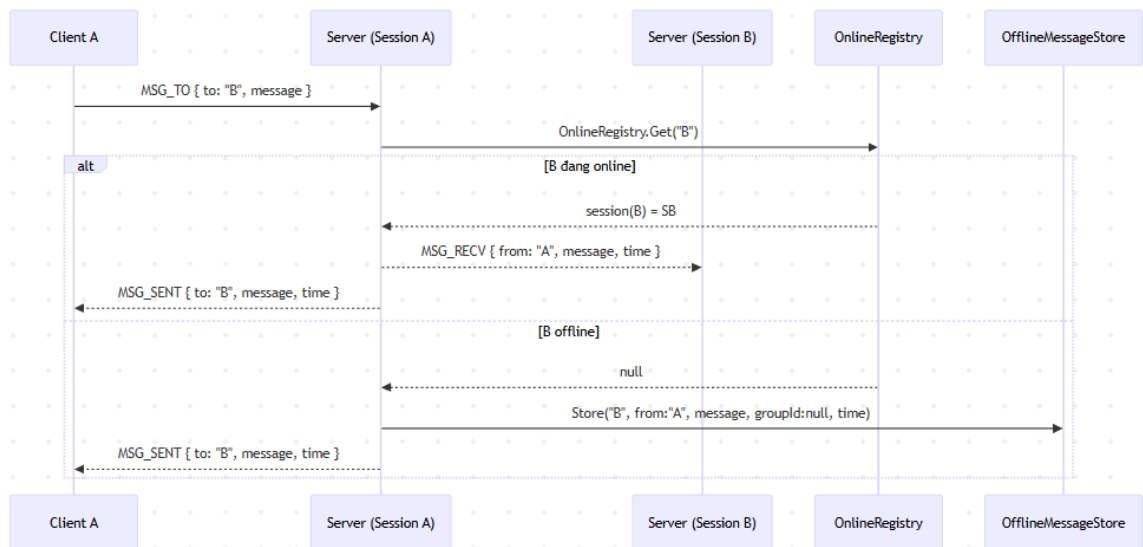
3. Server nhận chuỗi, tuần tự hóa chuỗi, lấy type, so sánh switch case, so sánh username trong lớp Online, xảy ra 2 trường hợp:

- Với Client B đang online:

+ Giả sử chuỗi đúng với format, tiến hành gửi đến Client B, và thông báo cho Client A đã gửi (hiển thị “bubble” tin nhắn giữa Client A và Client B)

- Với Client B đang offline:

+ Server tiến hành gọi lớp OfflineMessageStore (lớp lưu trữ tin nhắn offline), thực hiện lưu trữ tin nhắn. Tin nhắn sẽ được gửi đi cho Client B khi online.

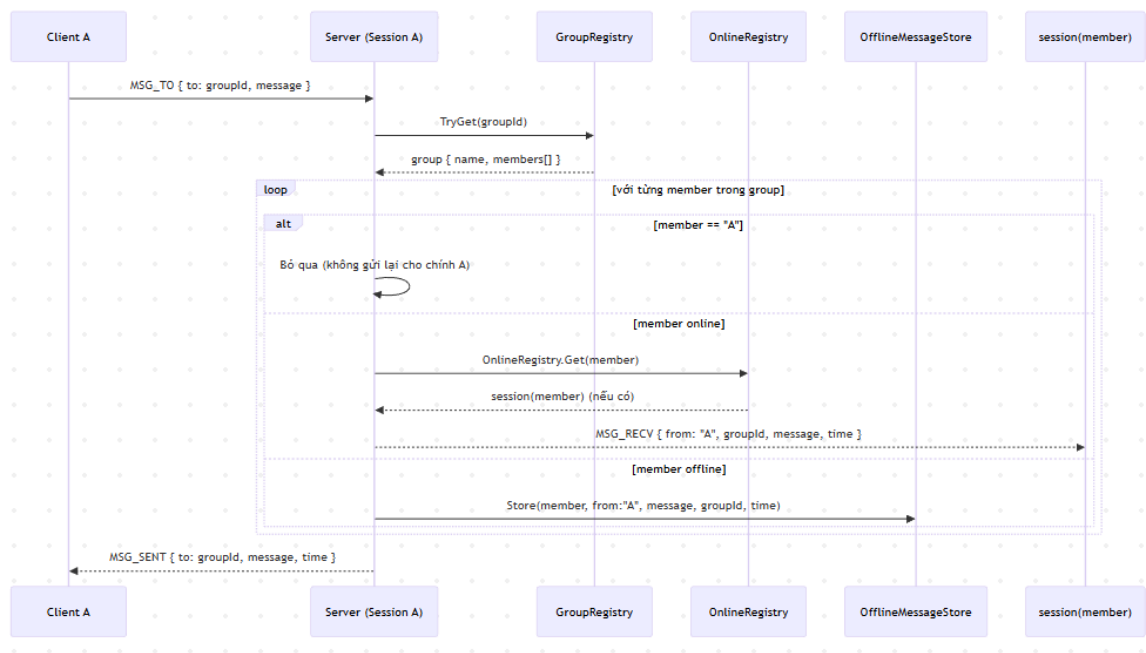


Hình 0-3 Luồng gửi tin nhắn tới một user

c. Luồng gửi tin nhắn tới một group

1. Cách thức vẫn như trên, đóng gói “MSG_TO_GROUP”, gửi lên server
2. Server nhận type “MSG_TO_GROUP”, tiến hành boardcast đến tất cả thành viên có trong GroupID đó:

- Nếu thành viên online: chuyển trực tiếp tin nhắn đến Client
- Nếu offline: lưu trữ tin nhắn ở phía server, sau khi Client online thì tiến hành gửi lại

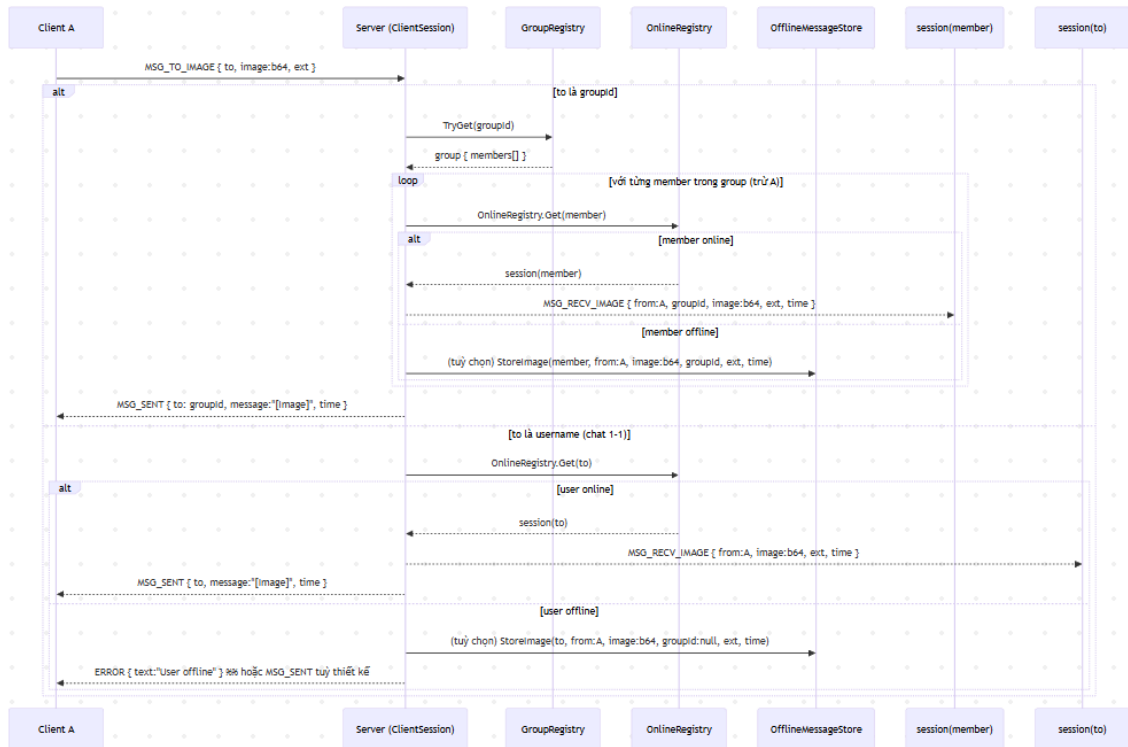


Hình 0-4 Luồng gửi tin nhắn tới một group

d. Luồng gửi ảnh

- Cách thức gửi hình ảnh vẫn như trên, chỉ khác so với tin nhắn như sau:

1. Image được convert sang một mảng byte[]
2. Mảng byte[] được convert thành base64
3. Khi đến destination, tiến hành convert ngược thành image.



Hình 0-5 Luồng gửi ảnh

CHƯƠNG 2: CÁC KỸ THUẬT ,GIAO THỨC SỬ DỤNG

2.1. Công cụ hỗ trợ

- Ngôn ngữ: C#
- Nền tảng: .NET Framework
- IDE: Visual Studio Community
- Giao diện: Winform, GunaUI

2.2 Mô hình kiến trúc

Hệ thống được triển khai theo kiến trúc Client–Server, trong đó:

Server:

- Quản lý các kết nối từ Client.
- Duy trì danh sách Client online và thông tin kết nối.
- Xử lý và phân phối tin nhắn (broadcast, trực tiếp, nhóm, offline).

Client:

- Gửi và nhận dữ liệu từ Server.
- Hiển thị giao diện chat, cho phép người dùng gửi tin nhắn, tham gia nhóm và nhận thông báo.

Kiến trúc này đảm bảo tính tập trung, đồng bộ và mở rộng, cho phép Server điều phối mọi hoạt động của hệ thống một cách hiệu quả.

2.3. Kiến trúc

1. Socket Programming (TCP/IP):

- Server và Client sử dụng TCP sockets để truyền dữ liệu.
- TCP đảm bảo dữ liệu được gửi đúng, theo thứ tự và đáng tin cậy, tránh mất mát hoặc sai thứ tự tin nhắn.

2. Đa luồng (Multithreading):

- Server tạo một thread riêng cho mỗi Client nhằm xử lý đồng thời nhiều kết nối mà không làm nghẽn hệ thống.
- Client có thể sử dụng thread riêng để lắng nghe tin nhắn từ Server, giúp GUI luôn mượt mà và không bị treo trong quá trình nhận dữ liệu.

2.6 Các kỹ thuật bổ sung

1. Serialization / Deserialization:

- Cho phép gửi các đối tượng Message thay vì chỉ truyền chuỗi plain text.
- Giúp dữ liệu được đóng gói, truyền tải và giải mã dễ dàng giữa Server và Client, đồng thời hỗ trợ mở rộng tính năng như lưu lịch sử tin nhắn hoặc truyền hình ảnh.

2. Lock / Monitor / Concurrent Collections:

- Được sử dụng để bảo vệ dữ liệu chung (ví dụ danh sách Client, nhóm chat) trong môi trường đa luồng.
- Đảm bảo tính nhất quán và an toàn dữ liệu, tránh xung đột khi nhiều thread cùng truy cập hoặc sửa đổi dữ liệu.

CHƯƠNG 3: MÔ TẢ CÁC LỚP VÀ PHƯƠNG THỨC CHÍNH

3.1 Lớp Server

Class: ChatServer

Mục đích: Quản lý kết nối của các Client, xử lý tin nhắn, duy trì danh sách Client online/offline, và quản lý nhóm chat.

Các thuộc tính chính:

- TcpListener server: lắng nghe các kết nối từ Client.
- List<ClientHandler> clients: danh sách các Client đang online.
- int port: cổng mà Server sử dụng để kết nối.

Các phương thức chính:

- Start()
 - Khởi động Server, bắt đầu lắng nghe kết nối mới.
 - Tạo thread riêng để chấp nhận và xử lý các Client kết nối đồng thời.
- BroadcastMessage(string message)
 - Gửi tin nhắn tới tất cả Client đang online.
- SendMessageToClient(string username, string message)
 - Gửi tin nhắn trực tiếp tới một Client cụ thể dựa trên username.
- CreateGroup(string groupName, List<string> members)
 - Tạo nhóm chat mới và lưu danh sách thành viên.
- SendGroupMessage(string groupName, string message)
 - Gửi tin nhắn đến tất cả thành viên trong nhóm.
- SaveOfflineMessage(string username, string message)
 - Lưu tin nhắn khi Client đích không online, để gửi khi họ kết nối lại.
- RemoveClient(ClientHandler client)
 - Xóa Client khỏi danh sách online khi họ thoát hệ thống.

3.2 Lớp Client Handler

Class: ClientHandler

3.2.1 Mục đích

ClientHandler đại diện cho một Client duy nhất đang kết nối tới Server.

Lớp này chịu trách nhiệm:

- Xử lý giao tiếp giữa Server và từng Client.
- Nhận và phân tích tin nhắn từ Client.
- Gửi phản hồi hoặc dữ liệu từ Server tới Client.
- Thông báo cho Server khi Client ngắt kết nối.

3.2.2 Các thuộc tính chính

TcpClient: client

- Đại diện kết nối TCP của Client.
- Dùng để đọc/ghi dữ liệu từ/đến Client.

NetworkStream: stream

- Luồng dữ liệu được mở khi Client kết nối thành công.
- Dùng để gửi & nhận message qua TCP.

String: username

- Tên đăng nhập của Client.
- Được thiết lập ngay khi Client thực hiện login.

ChatServer: server

Tham chiếu đến Server cha để gọi các chức năng:

- Broadcast
- Gửi tin nhắn chỉ định
- Lưu offline
- Xóa client khi disconnect

bool: isConnected

- Trạng thái kết nối để xử lý vòng lặp Listen().

3.2.3 Các phương thức chính

Listen()

- Chạy trong một thread riêng biệt cho mỗi Client.
- Liên tục lắng nghe tin nhắn từ Client qua stream.
- Phân tích loại message:
 - Đăng nhập
 - Gửi tin nhắn cá nhân
 - Gửi tin nhắn nhóm
 - Yêu cầu broadcast
 - Đăng xuất
- Nếu Client mất kết nối → báo về Server để xóa khỏi danh sách online.

SendMessage(string message)

- Gửi tin nhắn từ Server tới Client.
- Chuyển message sang byte[] và ghi xuống stream.
- Bảo đảm an toàn (try-catch) để phát hiện mất kết nối.

3.2.4 Luồng hoạt động ClientHandler

- 1 Server nhận kết nối mới → tạo ClientHandler.
 - 2 Server mở luồng → gọi Listen() trong thread mới.
 - 3 Client gửi thông điệp → Listen() đọc & phân tích.
 - 4 ClientHandler chuyển yêu cầu sang Server:
 - Gửi riêng
 - Gửi nhóm
 - Broadcast
 - Lưu offline
1. Khi Client ngắt kết nối:
 - Đóng stream
 - Đóng socket
 - Báo lại Server → gọi RemoveClient()

3.2.5 Vai trò trong toàn hệ thống

ClientHandler giống như “người quản lý” của từng Client:

- Quản lý giao thức giao tiếp TCP.
- Giữ thông tin người dùng.
- Kết nối trực tiếp với logic của Server.
- Đảm bảo mỗi Client hoạt động độc lập và không ảnh hưởng nhau.

3.3 Lớp Client

Class: ChatClient

3.3.1 Mục đích

ChatClient đại diện cho ứng dụng Client chạy trên máy người dùng (WinForms).

Lớp này chịu trách nhiệm:

- Kết nối đến Server qua TCP.
- Gửi và nhận tin nhắn từ Server.
- Cập nhật giao diện (UI) khi có tin nhắn mới.
- Gửi yêu cầu như: đăng nhập, gửi tin riêng, gửi nhóm, broadcast,...
- Quản lý việc ngắt kết nối.

3.3.2 Các thuộc tính chính

TcpClient: client

- Đại diện kết nối TCP đến Server.
- Dùng để thiết lập kênh trao đổi dữ liệu.

NetworkStream: stream

- Luồng dữ liệu phía Client dùng để gửi/nhận từ Server.

String: username

- Tên đăng nhập của người dùng.
- Gửi sang Server ngay khi Connect() để xác thực.

Thread: listenThread

- Thread chạy song song để lắng nghe tin nhắn từ Server liên tục.

Bool: isConnected

- Trạng thái kết nối hiện tại.

3.3.3 Các phương thức chính

Connect(string serverIP, int port)

Chức năng:

- Tạo TcpClient và kết nối tới server thông qua IP + port.
- Mở NetworkStream.
- Gửi thông tin đăng nhập (username) đến Server.
- Tạo và khởi chạy thread cho ReceiveMessage().

Quy trình:

- Gọi client.Connect(serverIP, port)
- Lấy stream = client.GetStream()
- Gửi JSON hoặc plain text: "LOGIN|username"
- Khởi chạy listenThread để nhận tin từ server.

SendMessage(string message)

Chức năng:

- Gửi tin nhắn từ Client lên Server.
- Dùng để:
 - Gửi tin nhắn thường
 - Gửi tin riêng (private message)
 - Gửi tin nhóm
 - Gửi broadcast

Hoạt động:

- Chuyển message sang byte[].
- Ghi xuống stream → Server xử lý.

ReceiveMessage()

Chức năng:

- Lắng nghe dữ liệu từ Server liên tục trong một thread riêng.
- Mỗi khi Server gửi tin:

- Đọc dữ liệu từ stream
- Phân tích nội dung
- Hiển thị lên giao diện WinForms (Invoke để tránh lỗi cross-thread)

Đảm bảo:

- Không chặn UI.
- Không làm treo giao diện khi stream bị delay.

Disconnect()

Chức năng:

- Gửi thông báo đăng xuất cho Server.
- Đóng stream và TcpClient.
- Dừng thread lắng nghe.
- Tránh lỗi “ghost client” trên Server.

Quy trình:

1. Gửi "LOGOUT" lên server.
2. stream.Close()
3. client.Close()
4. listenThread.Abort() hoặc flag STOP.
5. Cập nhật UI cho người dùng.

3.3.4 Luồng hoạt động của ChatClient

1. Người dùng mở ứng dụng → nhập Username.
2. Nhấn Connect → gọi Connect()
3. Client gửi LOGIN và nhận danh sách người online.
4. Khi người dùng gửi:
 - Tin nhắn chung → SendMessage()
 - Tin nhắn riêng → SendMessage("PRIVATE|A→B|Nội dung")
 - Gửi nhóm → SendMessage("GROUP|groupName|Nội dung")
5. Server phản hồi → ReceiveMessage() cập nhật UI.
6. Người dùng thoát app → Disconnect().

3.3.5 Vai trò trong toàn hệ thống

ChatClient đóng vai trò:

- Interface giữa người dùng và Server.
- Hiển thị thông tin chat bảo đảm realtime.
- Gửi yêu cầu và nhận phản hồi.
- Xử lý giao diện WinForms: danh sách online, nhóm, lịch sử,...

3.4 Lớp Message (tùy chọn)

Class: Message

3.4.1 Mục đích

Lớp Message dùng để đóng gói dữ liệu tin nhắn trong hệ thống Chat theo dạng đối tượng.

Việc sử dụng lớp này mang lại các lợi ích:

- Tăng tính rõ ràng và chuẩn hóa dữ liệu truyền đi.
- Dễ dàng mở rộng khi cần thêm loại tin nhắn mới.
- Hỗ trợ Serialization để gửi qua mạng và Deserialization khi nhận từ Server/Client.
- Hạn chế lỗi định dạng khi dùng string thuần (plain text).

3.4.2 Các thuộc tính

string Type

- Xác định loại tin nhắn.
- Ví dụ:
 - "broadcast" – gửi cho tất cả Client.
 - "direct" – gửi riêng 1 Client.
 - "group" – gửi trong nhóm.
 - "offline" – lưu để xử lý khi người nhận online.
 - "system" – server gửi thông báo.

string Sender

- Tên người gửi tin nhắn.

string Receiver

- Tên người nhận hoặc tên nhóm.
- Dùng khi loại tin nhắn là direct hoặc group.

string Content

- Nội dung tin nhắn dạng văn bản.

DateTime Timestamp

- Thời điểm tin nhắn được tạo.
- Hỗ trợ hiển thị lịch sử chat hoặc offline messages.

3.4.3 Các phương thức chính

Serialize()

Chức năng: Chuyển đối tượng Message thành chuỗi để gửi đi qua TCP.

Có thể serialize theo hai cách:

1. Dạng Custom Protocol:

“ TYPE|SENDER|RECEIVER|CONTENT|TIMESTAMP ”

2. JSON (Đề xuất – chuẩn, dễ quản lý hơn):

```
{  
  "Type": "direct",  
  "Sender": "Alice",  
  "Receiver": "Bob",  
  "Content": "Hello!",  
  "Timestamp": "2025-11-14T11:00:00"  
}
```

Ưu điểm JSON:

- Dễ đọc
- Dễ debug
- Dễ mở rộng

Deserialize(string data)

Chức năng: Chuyển chuỗi dữ liệu do Server/Client gửi đến thành đối tượng Message.

Có thể thực hiện:

- Parse từ custom protocol
- Hoặc dùng JSON deserialization

Ví dụ dùng JSON:

```
“ Message msg = JsonConvert.DeserializeObject<Message>(data); ”
```

3.4.4 Vai trò trong hệ thống

Lớp Message giúp:

- Chuẩn hóa toàn bộ tin nhắn trao đổi trong hệ thống.
- Dễ dàng thêm tính năng như:
 - gửi file
 - thông báo server
 - phản hồi lỗi
 - gửi trạng thái online/offline
- Giảm xử lý string thủ công trên Server và Client.
- Tăng tính mở rộng và bảo trì về lâu dài.

3.4.5 Vị trí của lớp Message trong luồng hoạt động

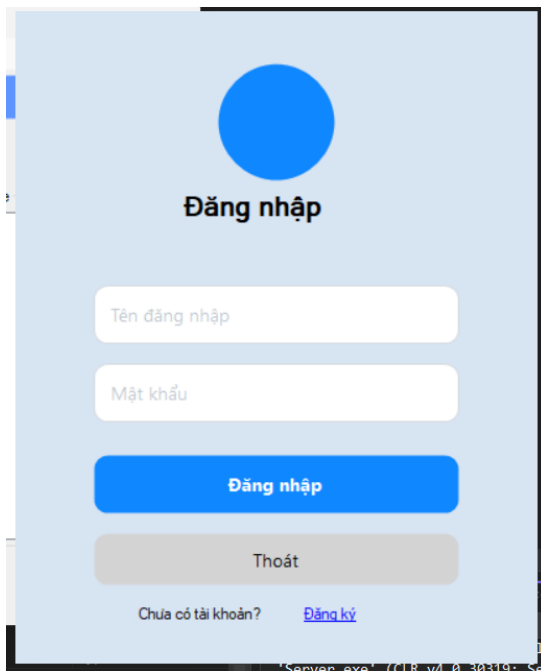
1. Client tạo một Message.
2. Gọi Serialize() để chuyển thành chuỗi.
3. Chuỗi được gửi qua TCP đến Server.
4. Server nhận chuỗi → Deserialize() thành Message.
5. Server xử lý → tạo Message phản hồi.
6. Server gửi lại → Client Deserialize.
7. Client hiển thị nội dung lên giao diện.

CHƯƠNG 4: DEMO

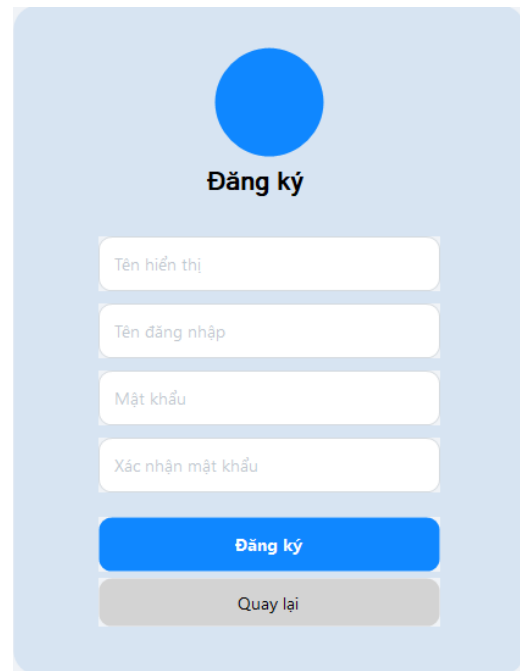
4.1 Phía người dùng:

4.1.1 Giao diện đăng nhập và đăng ký:

Đây là cửa sổ đầu tiên khi một người dùng mở ứng dụng, có yêu cầu tên đăng nhập, mật khẩu, có liên kết đến form đăng kí cho trường hợp người dùng chưa có tài khoản (hình 4.1)



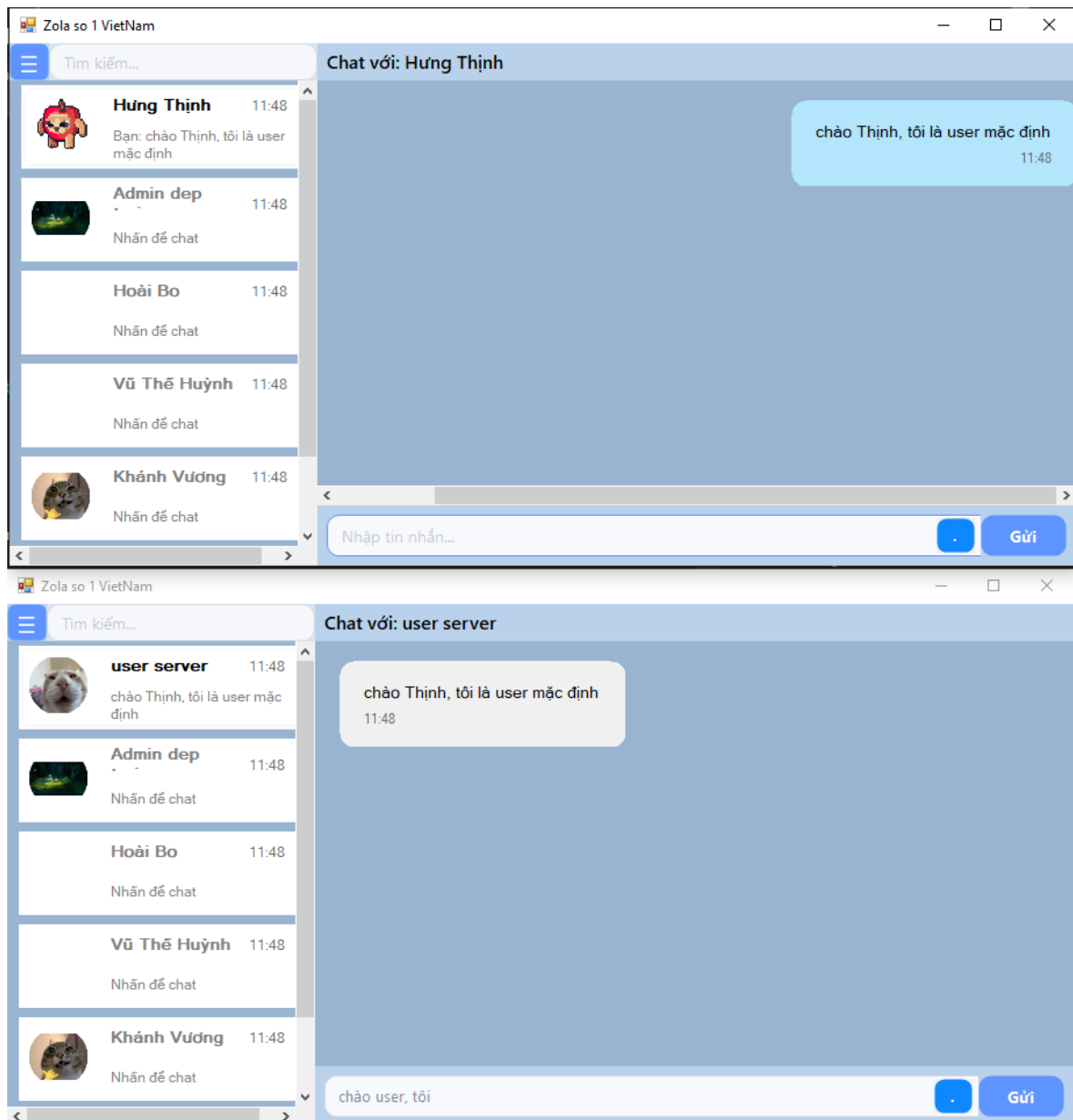
Hình 0-1 . Form đăng nhập



Hình 0-2 . Form đăng ký

4.1.2 Gửi tin nhắn trực tiếp giữa các Client:

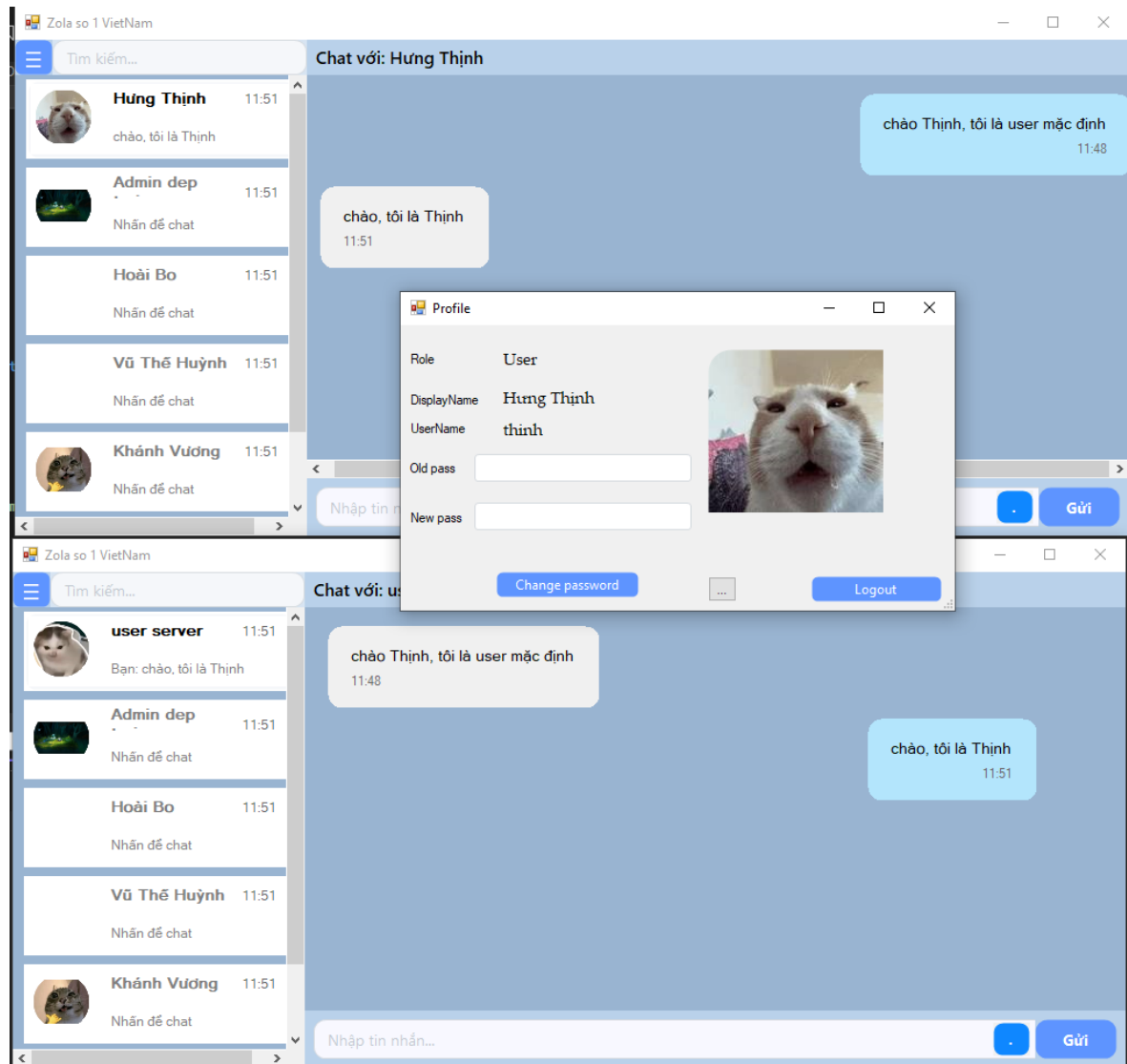
Đây là cửa sổ chính sau khi đăng nhập thành công. Bên trái hiển thị danh sách các tài khoản đang có trong hệ thống. Mỗi người dùng được hiển thị với ảnh đại diện, tên và tin nhắn gần nhất. Khi người dùng chọn một tài khoản bất kỳ, cửa sổ chat phía bên phải tương ứng sẽ được mở. (hình 4.2)



Hình 0-3 Hình ảnh gửi tin nhắn giữa client - client

4.1.3 Xem hồ sơ người dùng:

Sau khi nhấn vào nút ba gạch sẽ được tùy chỉnh hồ sơ người dùng cho phép thay đổi tên, mật khẩu, ảnh đại diện,... (hình 4.3)

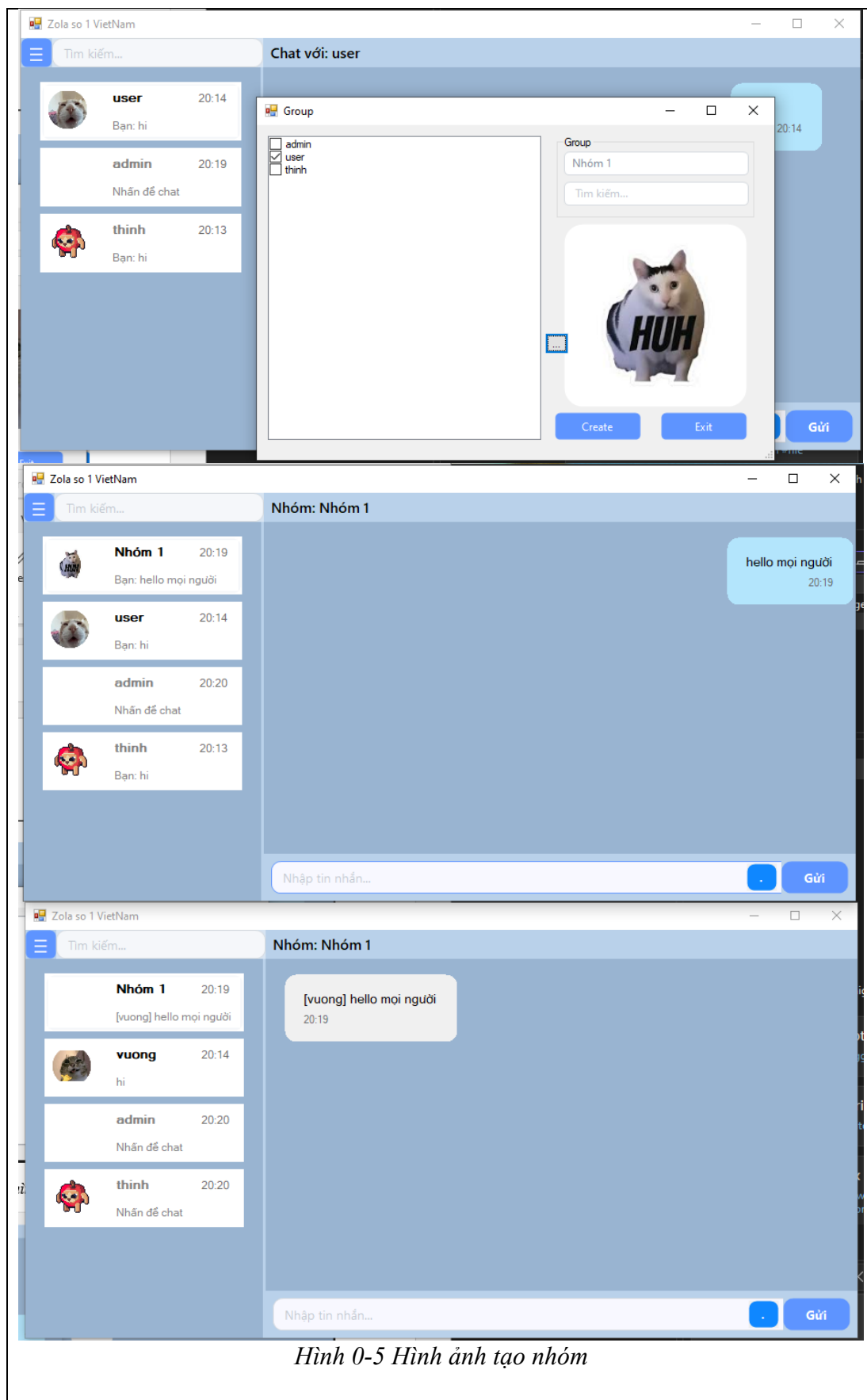


Hình 0-4 Form profile

4.1.4 Tạo nhóm chat:

- Sau khi người dùng nhấn vào phần tạo nhóm chat sẽ hiển thị form tạo nhóm gồm thêm các thành viên, chọn nhóm cộng đồng hoặc nhóm thường (hình 4.4), sau khi hoàn thành các bước trên ta sẽ được đưa đến giao diện nhóm (hình 4.5)

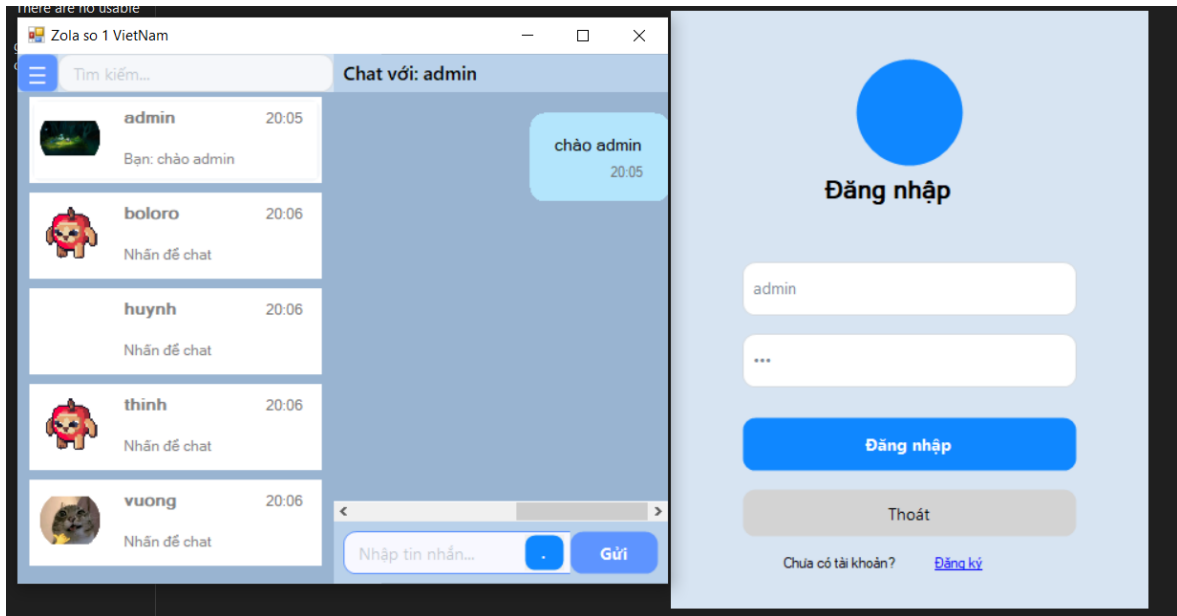
- Phần này bao gồm chat offline, nếu các Client trong nhóm offline, thì tin nhắn sẽ được lưu trữ ở phía server. Sau khi người dùng online, server bắt đầu gửi lại tin nhắn trong lúc người dùng offline



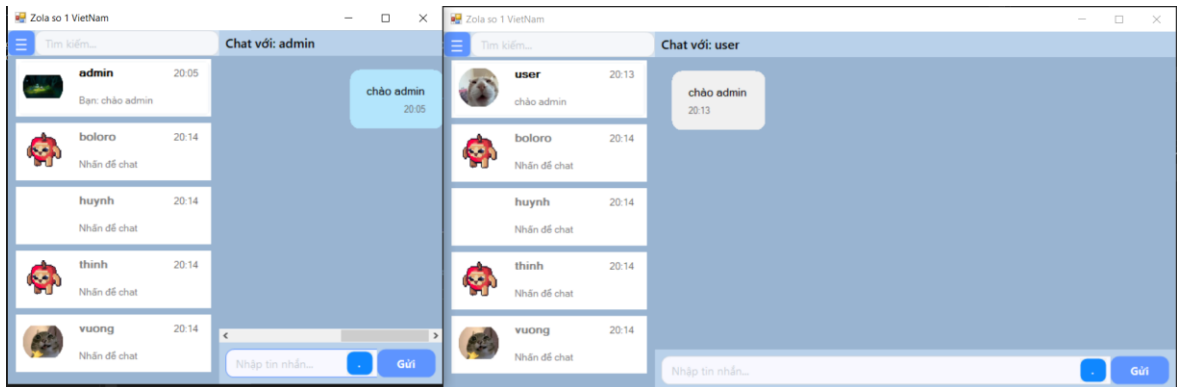
Hình 0-5 Hình ảnh tạo nhóm

4.1.5 Nhắn tin khi người dùng offline:

Khi gửi tin nhắn đến một người dùng chưa kết nối với server chương trình sẽ lưu tin nhắn lên server kèm theo thời gian và người nhận sẽ nhận đc sau khi kết nối với server



Hình 0-6 Trước khi người nhận đăng nhập



Hình 0-7 Sau khi người nhận đăng nhập

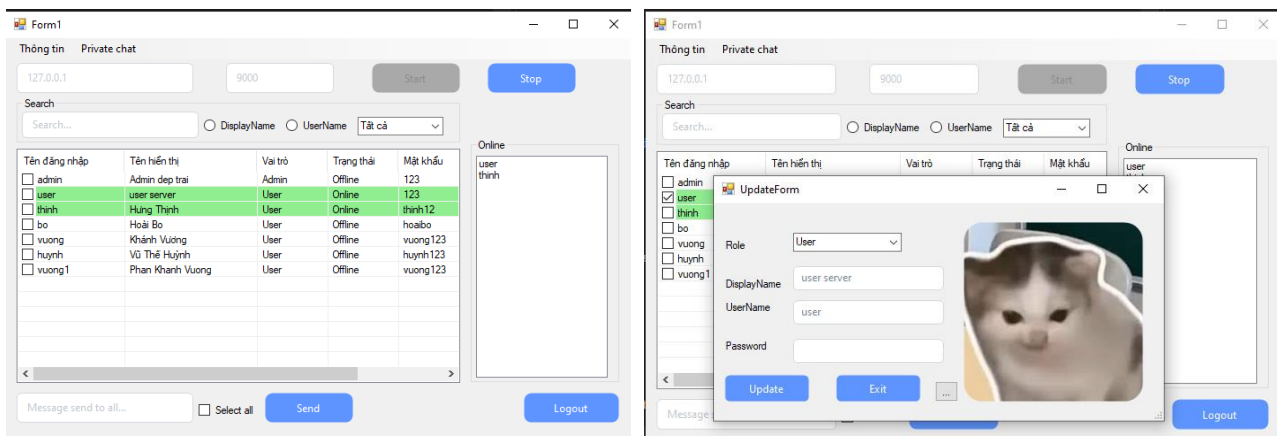
4.2 Phía máy chủ (server):

4.2.1 Giao diện đăng nhập:

Giao diện đăng nhập sẽ tương tự như giao diện người dùng (hình 4.1)

4.2.2 Giao diện quản lý danh sách client :

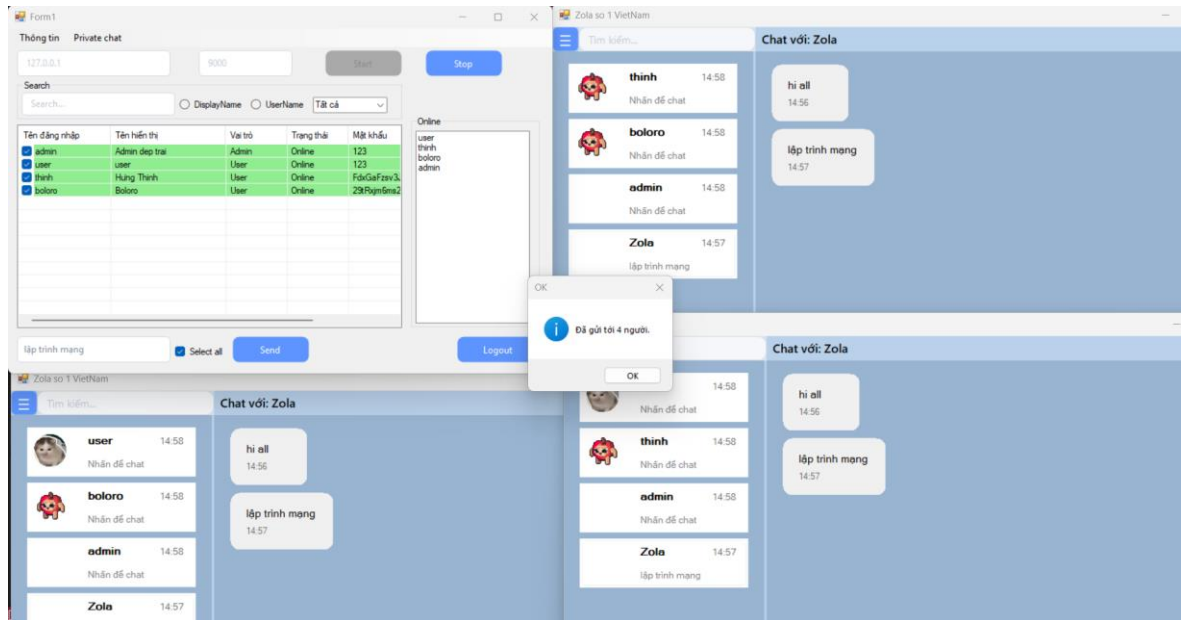
Giao diện cho phép admin nhập IP và Port để bật/tắt server. Bên dưới là bảng hiển thị danh sách tài khoản gồm tên đăng nhập, tên hiển thị, vai trò, trạng thái và mật khẩu. Admin có thể tìm kiếm theo tên hiển thị hoặc username, lọc theo vai trò và chọn nhiều user để gửi tin nhắn đến các user đã chọn. Khung bên phải hiển thị danh sách người dùng đang online. Nút Logout dùng để thoát khỏi giao diện quản lý. (hình 4.6)



Hình 0-8 Form quản lý client (server)

4.2.3 Chức năng gửi tin nhắn cho tất cả Client:

Giao diện cho phép admin chọn một hoặc nhiều tài khoản trong danh sách rồi nhập nội dung tin nhắn ở ô phía dưới. Khi nhấn Send, server sẽ gửi thông báo đến tất cả client đã chọn. Sau khi gửi, hệ thống hiển thị hộp thoại thông báo số lượng người nhận. Tin nhắn sẽ xuất hiện trên cửa sổ chat của các client tương ứng. (hình 4.7)



Hình 0-9 Gửi tin nhắn cho toàn bộ client

CHƯƠNG 6: KẾT QUẢ ĐẠT ĐƯỢC

6.1. Kết quả triển khai các chức năng cơ bản

- Về cơ bản, nhóm đã hoàn thành hơn 95% nhiệm vụ cơ bản được giao.

6.1.1. Khả năng đa kết nối Client và đăng nhập xác thực.

Hệ thống Server đã được triển khai với cơ chế Đa Luồng (Multi-threading) trên nền tảng Socket, cho phép Server quản lý và duy trì đồng thời nhiều kết nối Client mà không bị chặn (non-blocking I/O).

- Xác thực Bắt buộc: Mỗi Client chỉ được phép thiết lập kênh giao tiếp hoàn chỉnh sau khi hoàn tất quá trình Đăng nhập và Xác thực thông qua Cơ sở Dữ liệu. Điều này đảm bảo mỗi kết nối socket được gán với một ID Người dùng hợp lệ.

6.1.2. Quản lý Trạng thái Client (Presence Management)

Server duy trì một cấu trúc dữ liệu để theo dõi danh sách người dùng đang trực tuyến (Online): Khi Client đăng nhập thành công, Server lưu trữ ánh xạ giữa ID Người dùng và Đối tượng Socket của họ, cho phép server theo dõi trạng thái hoạt động của tập các Client.

6.1.3. Cơ chế trao đổi tin nhắn Server-Client

Ứng dụng đã thiết lập một giao thức tin nhắn dựa trên định dạng JSON để chuẩn hóa các gói tin trao đổi.

- Truyền thông Hai chiều: Cho phép Client gửi tin nhắn yêu cầu đến Server và Server gửi phản hồi hoặc thông báo đến các Client.
- Gửi Thông báo Toàn bộ (Broadcast): Server có khả năng gửi cùng một thông điệp đến tất cả các Client đang trực tuyến bằng cách lặp qua danh sách các đối tượng Socket đang hoạt động. Chức năng này được sử dụng hiệu quả để gửi các thông báo hệ thống quan trọng.

6.2. Kết quả triển khai các yêu cầu nâng cao

Nhóm đã hoàn thành hơn 95% công việc được giao.

6.2.1. Gửi tin nhắn trực tiếp giữa các client

Chức năng này được thực hiện thông qua Server như một bộ định tuyến trung tâm.

- Logic Định tuyến: Khi Client (A) gửi tin nhắn, Server nhận và trích xuất ID người nhận (B). Server tra cứu ID này trong danh sách Client đang Online.

- Chuyển tiếp: Nếu tìm thấy kết nối của Client B, Server sẽ chuyển tiếp gói tin tức thời. Cơ chế này giúp giảm thiểu độ trễ cho các cuộc trò chuyện cá nhân.

6.2.2. Tạo nhóm chat

Hệ thống đã xây dựng mô hình dữ liệu và logic để quản lý các nhóm trò chuyện.

- Quản lý Nhóm: Server lưu trữ thông tin ID Nhóm, Tên Nhóm và Danh sách Thành viên trong Cơ sở Dữ liệu.
- Phân phối Tin nhắn Nhóm: Khi nhận được tin nhắn nhóm, Server thực hiện việc truy vấn danh sách thành viên của nhóm. Sau đó, Server phân phối tin nhắn đến tất cả các thành viên đang online của nhóm.

6.2.3. Gửi tin nhắn ngoại tuyến

Đây là tính năng quan trọng nhằm đảm bảo tin nhắn không bị mất khi người nhận không kết nối.

- Lưu trữ Lịch sử: Mọi tin nhắn (cá nhân và nhóm) đều được lưu trữ vào bảng Lịch sử Tin nhắn trong json, kèm theo trạng thái gửi.
- Xử lý Ngoại tuyến: Nếu Server không tìm thấy người nhận đang trực tuyến, tin nhắn sẽ được chờ.
- Đồng bộ khi trực tuyến: Khi người dùng ngoại tuyến đăng nhập trở lại, Server sẽ tự động truy vấn tất cả tin nhắn "Chờ Gửi" dành cho người dùng đó, gửi chúng đi, và cập nhật trạng thái tin nhắn thành "Đã gửi".

6.3. Tổng kết

Hạng Mục	Chi Tiết
Thành tựu Chung	Nhóm đã hoàn thành hơn 95% các yêu cầu chức năng cơ bản và nâng cao đã đề ra.
Thách thức & Hạn chế	Còn tồn tại một số lỗi tiềm ẩn (minor bugs) chưa được kiểm tra và xử lý triệt để trong quá trình thử nghiệm.

TÀI LIỆU THAM KHẢO

- [1] Microsoft, "TcpClient Class (System.Net.Sockets)," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.tcpclient>. [Accessed: Dec. 9, 2025].
- [2] Microsoft, "TcpListener Class (System.Net.Sockets)," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.tcplistener>. [Accessed: Dec. 9, 2025].
- [3] Microsoft, "Asynchronous Server Socket Example," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/framework/network-programming/asynchronous-server-socket-example>. [Accessed: Dec. 9, 2025].
- [4] R. Blum, C# Network Programming, Alameda, CA, USA: Sybex, 2003, pp. 150-200.
- [5] S. Cleary, Concurrency in C# Cookbook: Asynchronous, Parallel, and Multithreaded Programming, 2nd ed., Sebastopol, CA: O'Reilly Media, 2019.
- [6] J. Postel, "Transmission Control Protocol," IETF, RFC 793, Sep. 1981. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc793>.